NAQLA

المختبر 1

المختبر 1 — بناء هيكل شحنة

هندسة الواجهات الخلفية بالوكلاء الذكيين

هندسة الواجهات الخلفية بالوكلاء الذكيين – بناء أنظمة خلفية قابلة للتوسع بالذكاء الاصطناعي ومن أجله

المختبر 1 — بناء هيكل شحنة

نظرة عامة

الهدف

بنهاية اليوم سيكون لديك تطبيق Laravel 12 يعمل لصالح **«شحنة» للخدمات اللوجستية** (Shahna Logistics)، وهي شركة شحن ليبية، مع نموذج النطاق (Domain Model) الكامل في PostgreSQL: المدن، والمسافات، وشركات النقل (Carriers)، والمركبات، وعروض الأسعار (Quotes)، والشحنات، وأحداث الشحنات. ستكون قد بنيته *بالاستعانة* بـ Claude Code، وستكون قد تحقّقت من كل سطر أنتجه الوكيل.

يدور اليوم حول عادتين ستستخدمهما طوال بقية الأسبوع:

1. **استكشف ← خطّط ← نفّذ ← تحقّق (Explore → Plan → Implement → Verify).** لا يكتب الوكيل أي شيفرة قبل أن تقرأ خطة وتوافق عليها.
2. **أنت المهندس.** الوكيل شريك سريع يخطئ أحياناً وهو واثق تماماً. مهمتك أن تكتشف ذلك.

ما ستبنيه

الجزء	مكانه
خطة النطاق مكتوبة مع الوكيل	`docs/plans/day1-domain.md`
ملفات ترحيل (Migrations) لسبعة جداول	`database/migrations/`
نماذج Eloquent مع العلاقات والتحويلات (Casts)	`app/Models/`
تعدادات مدعومة (Backed Enums) للحالة، ونوع المركبة، وحالة المركبة	`app/Enums/`
`ShipmentStatus::canTransitionTo()` مع اختبار وحدة (Unit Test) بـ Pest	`app/Enums/ShipmentStatus.php` ، `tests/Unit/`
مصانع (Factories) لكل نموذج	`database/factories/`
مُغذِّ (Seeder) يضم 8 مدن ليبية، والمسافات، وشركات النقل، والمركبات	`database/seeders/`

نقطة البداية

```
git clone https://github.com/OthmanHaba/shahna-api.git
cd shahna-api
git checkout day1-start
git switch -c my-day1
```

يحتوي الوسم (Tag) `day1-start` على تطبيق Laravel 12 جديد مثبّت فيه Sail (PostgreSQL 16 + Redis 7)، وPest، وSanctum، وLaravel Boost، إضافة إلى ملف `CLAUDE.md` يحوي أعراف المشروع، والملف `docs/requirements.md` (موجز منتج شحنة)، ومجلد `docs/plans/` فارغ.

توزيع الوقت

الجزء	الخطوة	الدقائق
الجزء 1 (11:00-12:30)	1. تشغيل التطبيق باستخدام Sail	20
	2. ربط Boost بـ Claude Code	10
	3. استكشاف قاعدة الشيفرة مع الوكيل	15
	4. قراءة الموجز وكتابة الخطة	30
	نقطة التحقق 1	15
الجزء 2 (14:15-15:45)	5. التعدادات، يدوياً	20
	6. `canTransitionTo` بالاختبار أولاً	15
	7. ملفات الترحيل والنماذج مع الوكيل	25
	8. المصانع والمُغذّي	20
	نقطة التحقق 2 والإيداع	10

نصيحة

إذا تأخرت عن الركب فلا داعي للذعر، ولا تتخطَّ التحقق لكي «تلحق». يمكنك في أي وقت الانتقال إلى الحل المرجعي بالأمر `git checkout day1-end` . إنجاز خطوات أقل بإتقان أفضل من إنجازها كلها دون وعي.

الجزء 1 – التشغيل، والاستكشاف، والتخطيط (11:00–12:30)

الخطوة 1 – تشغيل التطبيق باستخدام Sail (20 دقيقة)

ماذا ولماذا. يشغّل Laravel Sail كلاً من PHP وPostgreSQL وRedis داخل Docker، فلا تحتاج إلى تثبيت PHP على حاسوبك. كل أمر في هذه الدورة يعمل *داخل* حاوية (Container) Sail. تُنجَز هذه الخطوة **يدوياً**، دون الوكيل: يجب أن تفهم كيف تبدأ بيئتك قبل أن تسمح لأي شيء آخر بالمساس بها.

> **تحذير**
>
> **يدوياً.** لا تطلب من Claude Code تنفيذ هذه الخطوة. إذا كانت بيئتك معطّلة فسيُصلحها الوكيل بطرق لن تستطيع تفسيرها لاحقاً.

ثبّت اعتماديات PHP (Dependencies) باستخدام حاوية Composer مؤقتة (تحتاج إلى ذلك مرة واحدة فقط، لأن `vendor/` لا يُودَع في المستودع):

```
docker run --rm \
  -u "$(id -u):$(id -g)" \
  -v "$(pwd):/var/www/html" \
  -w /var/www/html \
  laravelsail/php84-composer:latest \
  composer install --ignore-platform-reqs
```

أنشئ ملف البيئة وأضف اسماً مستعاراً (Alias) في الصدفة (Shell) لتكتب `sail` بدلاً من `./vendor/bin/sail`:

```
cp .env.example .env
alias sail='sh $([ -f sail ] && echo sail || echo vendor/bin/sail)'
```

أضف سطر الاسم المستعار إلى `~/.zshrc` أو `~/.bashrc` ليبقى متاحاً عند فتح طرفية جديدة.

شغّل الحاويات، وولّد مفتاح التطبيق، ونفّذ ملفات الترحيل:

```
sail up -d
sail artisan key:generate
sail artisan migrate
```

تحقّق من أن كل شيء يعمل:

```
sail ps
```

المخرجات المتوقعة (مع اختصار الأعمدة):

```
NAME                      SERVICE    STATUS
shahna-api-laravel.test-1   laravel.test   Up
shahna-api-pgsql-1          pgsql          Up (healthy)
shahna-api-redis-1          redis          Up (healthy)
```

افتح `http://localhost` في المتصفح: يُفترض أن ترى صفحة الترحيب في Laravel. ثم شغّل مجموعة الاختبارات:

```
sail artisan test
```

```
   PASS  Tests\Unit\ExampleTest
  ✓ that true is true

   PASS  Tests\Feature\ExampleTest
  ✓ the application returns a successful response

  Tests:    2 passed (2 assertions)
```

افحص ملف `.env`. افتحه وتأكّد من وجود هذه الأسطر. يجب أن تكون قادراً على شرح كل واحد منها.

```
DB_CONNECTION=pgsql
DB_HOST=pgsql
DB_PORT=5432
DB_DATABASE=shahna
REDIS_HOST=redis
```

القيمة `DB_HOST=pgsql` ليست خطأً مطبعياً: داخل Docker تصل الحاويات بعضها إلى بعض باسم الخدمة، لا عبر `localhost`.

> **تحذير**
>
> **أعطال شائعة.** المنفذ 80 مستخدم مسبقاً: اضبط `APP_PORT=8080` في `.env` واستخدم `http://localhost:8080`. المنفذ 5432 مستخدم (نسخة PostgreSQL محلية): اضبط `FORWARD_DB_PORT=54320`. رسالة «Permission denied» على `vendor/`: لقد تخطّيت الخيار `-u`؛ احذف `vendor/` وأعد تشغيل الأمر.

الخطوة 2 — ربط Laravel Boostg Claude Code (10 دقائق)

ماذا ولماذا. Laravel Boost خادم MCP (MCP Server) يزوّد الوكيل بمعلومات *حقيقية* عن تطبيقك: إصدارات الحزم المثبّتة، ومخطط قاعدة البيانات (Schema)، والمسارات (Routes)، والسجلات (Logs)، وتوثيق Laravel الخاص بالإصدار المستخدم. من دونه يخمّن الوكيل اعتماداً على بيانات تدريبه، التي قد تصف Laravel 9 أو واجهة برمجة لحزمة لم توجد قط.

ثبّت تكامل Boost مع Claude Code لهذا المشروع. يسألك المثبّت عن الوكلاء الذين تستخدمهم: اختر Claude Code. يكتب المثبّت الملف `.mcp.json` (الذي يشغّل خادم MCP بالأمر `artisan boost:mcp` عبر Sail) ويضيف إرشادات Boost إلى نهاية `CLAUDE.md`:

```
sail artisan boost:install
```

راجع الفرق (Diff) في `CLAUDE.md` باستخدام `git diff CLAUDE.md` قبل أن تودعه.

شغّل Claude Code من جذر المشروع:

```
claude
```

داخل الجلسة، اكتب `/mcp`. يُفترض أن ترى `laravel-boost` مدرجاً بحالة **connected**. إذا ظهرت الحالة **failed**، فتحقّق من أن `sail ps` يُظهر الحاويات قيد التشغيل، ثم أعد تشغيل Claude Code.

اقرأ الآن `CLAUDE.md` بنفسك (في محرّرك، لا عبر الوكيل). يُحمَّل هذا الملف في سياق الوكيل في كل جلسة. وهو يخبر الوكيل بأعراف المشروع: Laravel 12، Pestg، PostgreSQL، والمبالغ المالية بالنوع `decimal(10,2)`، وتشغيل الأوامر عبر `sail`، والفروق الصغيرة، والاختبارات مع كل تغيير.

> **نصيحة**
>
> كل رمز (Token) في `CLAUDE.md` تدفع ثمنه مع كل طلب. اجعله قصيراً ومحدداً. عبارة «اكتب شيفرة نظيفة» عديمة الفائدة؛ أما «Money columns are `decimal(10,2)`, never float» فتغيّر سلوك الوكيل فعلاً.

الخطوة 3 – استكشاف قاعدة الشيفرة مع الوكيل (15 دقيقة)

ماذا ولماذا. المرحلة الأولى من الحلقة هي **الاستكشاف** (Explore). قبل أن تطلب أي تغيير، اجعل الوكيل يقرأ المشروع ويقدّم لك تقريراً. هذا يملأ سياقه بحقائق عن قاعدة الشيفرة *هذه* بدلاً من معرفة Laravel العامة، ويتيح لك التأكد مما إذا كان الوكيل يفهم فعلاً ما هو موجود.

🤖 اسأل وكيلك الذكي

طلب للوكيل

لا تغيّر أي ملفات. استكشف هذا المستودع وقدّم لي تقريراً قصيراً: 1. إصدار Laravel، وإصدار PHP، وإصدارات Pest Sanctumg Boostg (استخدم أدوات Boost، لا التخمين). 2. ما قاعدة البيانات وذاكرة التخزين المؤقت المضبوطتان، وأين ضُبطتا. 3. ما الجداول الموجودة حالياً (استخدم أداة مخطط قاعدة البيانات في Boost). 4. ما المسارات الموجودة (استخدم أداة قائمة المسارات في Boost). 5. ملخص للملف docs/requirements.md في 5 نقاط. 6. أي شيء في CLAUDE.md يجب أن أعرفه قبل أن نكتب الشيفرة. اذكر مسار الملف لكل ادعاء.

بالإنجليزية:

```
Do not change any files. Explore this repository and give me a short report:
1. Laravel version, PHP version, and the versions of Pest, Sanctum and Boost
   (use the Boost tools, not guesses).
2. Which database and cache are configured, and where that is set.
3. What tables exist right now (use the Boost database schema tool).
4. What routes exist (use the Boost route list tool).
5. A summary of docs/requirements.md in 5 bullet points.
6. Anything in CLAUDE.md I should know before we write code.
Cite the file path for every claim.
```

ما الذي تفحصه في مخرجات الوكيل.

- جيد: يستدعي أدوات Boost (سترى في السجل استدعاءات أدوات مثل `application-info` و`database-schema` و`list-routes`) ويذكر أن الإصدار **12.x** Laravel.
- جيد: لكل ادعاء مسار ملف يمكنك فتحه والتأكد منه.
- علامة خطر: يقول «Laravel 11» أو «MySQL» دون مصدر. اسأله: «من أين قرأت ذلك؟»
- علامة خطر: يعدّل ملفاً. لقد قلت «لا تغيّر أي ملفات». اضغط `Esc` لمقاطعته، ثم ارفض التغيير.

افتح بنفسك ملفين من الملفات المذكورة وتأكّد من صحة الادعاءات.

الخطوة 4 — قراءة الموجز وكتابة خطة النطاق (30 دقيقة)

ماذا ولماذا. هذه مرحلة **التخطيط** (Plan). الخطة المكتوبة رخيصة التغيير؛ أما ملف الترحيل الذي نُشر بالفعل فليس كذلك. ستُنتج الملف `docs/plans/day1-domain.md`، وهو مستند قصير يصف كل جدول وأعمدته وأنواعها وقيودها (Constraints) وعلاقاتها.

أولاً، **يدوياً**، اقضِ 5 دقائق في قراءة `docs/requirements.md` واكتب على الورق ثلاثة أسئلة لديك عن النطاق (مثلاً: «هل يمكن أن توجد شحنة دون عرض سعر؟»).

حوّل الآن Claude Code إلى **وضع التخطيط** (Plan Mode) (اضغط `Shift+Tab` حتى يُظهر شريط الحالة *plan mode*). في وضع التخطيط يستطيع الوكيل القراءة لكن لا يستطيع التعديل.

🤖 اسأل وكيلك الذكي

طلب للوكيل

اقرأ docs/requirements.md وCLAUDE.md. نحن نصمّم نموذج النطاق لليوم الأول فقط. اكتب خطة للملف docs/plans/day1-domain.md تغطي هذه النماذج، ولا شيء غيرها:

City (code، name_ar، name_en – واحد من GHA، SRT، TOB، ZAW، SEB، MRA، BEN، TIP) CityDistance (from_city_id، to_city_id، km) Carrier (name، phone) Vehicle (plate، type، تعداد van|box_truck|flatbed|reefer، capacity_kg، status، تعداد available|busy|maintenance، carrier_id) Quote (user_id، origin_city_id، destination_city_id، weight_kg، vehicle_type، (current_city_id يقبل القيمة الفارغة، price_lyd decimal(10,2)، expires_at) Shipment (reference بالصيغة "SHN-" + 6 حروف/أرقام كبيرة، user_id، quote_id فريد، vehicle_id، origin_city_id، destination_city_id، weight_kg، (vehicle_type، price_lyd decimal(10,2)، status، pickup_at ShipmentEvent (shipment_id، status، note، occurred_at)

أضف أيضاً عمود role إلى users: تعداد customer|dispatcher|admin، والقيمة الافتراضية customer.

لكل جدول اذكر: الأعمدة بأنواع PostgreSQL، وقابلية القيمة الفارغة، والمفاتيح الأجنبية مع سلوك الحذف (on-delete)، وقيود التفرّد، والفهارس، مع سطر تبرير واحد لكل منها. اذكر التعدادات بوصفها تعدادات PHP مدعومة في App. اذكر الترتيب الذي يجب أن تُنفَّذ به ملفات الترحيل. لا تُضِف جداول أو أعمدة أو حزماً أو ميزات غير مذكورة هنا. اختم بقائمة من الأسئلة المفتوحة الموجهة إليّ.

بالإنجليزية:

```
Read docs/requirements.md and CLAUDE.md. We are designing the Day 1 domain
model only. Write a plan for docs/plans/day1-domain.md covering these models,
and nothing else:

City (name_en, name_ar, code — one of TIP, BEN, MRA, SEB, ZAW, TOB, SRT, GHA)
CityDistance (from_city_id, to_city_id, km)
Carrier (name, phone)
Vehicle (carrier_id, plate, type enum van|box_truck|flatbed|reefer,
         capacity_kg, status enum available|busy|maintenance, current_city_id)
Quote (user_id, origin_city_id, destination_city_id, weight_kg, vehicle_type,
       price_lyd decimal(10,2), expires_at)
Shipment (reference "SHN-" + 6 uppercase alphanumerics, user_id,
          quote_id unique, vehicle_id nullable, origin_city_id,
          destination_city_id, weight_kg, vehicle_type, price_lyd decimal(10,2),
          status, pickup_at)
ShipmentEvent (shipment_id, status, note, occurred_at)

Also add a role column to users: enum customer|dispatcher|admin, default customer.

For every table list: columns with PostgreSQL types, nullability, foreign keys
with on-delete behaviour, unique constraints, and indexes, with one line of
justification each. List the enums as PHP backed enums in App\Enums.
List the order the migrations must run in.
Do NOT add tables, columns, packages or features that are not listed here.
End with a list of open questions for me.
```

ما الذي تفحصه في الخطة.

الفحص	جيد	علامة خطر
المبالغ المالية	`price_lyd numeric(10,2)`	`float` أو `double` أو عدد صحيح دون تفسير
مرجع الشحنة	`string` ، فريد، مفهرس	استخدام المعرّف ذي الزيادة التلقائية مرجعاً عاماً
`quote_id` في الشحنات	مفتاح أجنبي (Foreign Key) فريد	غير فريد (قد يُحجز عرض السعر نفسه مرتين)
رموز المدن	`char(3)` فريد أو نص قصير	لا يوجد قيد تفرّد
`CityDistance`	فريد على `(from_city_id, to_city_id)`	التكرار ممكن
النطاق	النماذج المذكورة بالضبط	جداول مختلَقة مثل `Payment` و`Invoice` و`Driver` و`Address`
التعدادات	`ShipmentStatus` ، `VehicleType` ، `VehicleStatus` ، `UserRole`	نصوص عادية دون تعداد

قارن الأسئلة المفتوحة التي طرحها الوكيل بالأسئلة الثلاثة التي كتبتها يدوياً. أجب عنها في المحادثة (أمثلة: «تُخزَّن المسافات في الاتجاهين»، «حذف شركة النقل ممنوع إذا كانت لديها مركبات»، «weight_kg عدد صحيح بلا إشارة»).

عندما تكون راضياً، اخرج من وضع التخطيط (`Shift+Tab`) واطلب:

🤖 **اسأل وكيلك الذكي**

طلب للوكيل

اكتب الخطة المتفق عليها في docs/plans/day1-domain.md، متضمنة إجاباتي عن الأسئلة المفتوحة. لا تنشئ أي ملفات أخرى.

بالإنجليزية:

```
Write the agreed plan to docs/plans/day1-domain.md, including my answers
to the open questions. Do not create any other files.
```

افتح الملف واقرأه من أوله إلى آخره. عدّل **يدوياً** أي شيء لا توافق عليه. أصبح هذا المستند الآن العقد الذي يحكم بقية اليوم.

```
git add docs/plans/day1-domain.md
git commit -m "docs: day 1 domain plan"
```

نقطة تحقق

نقطة التحقق 1 (12:30)

يُظهر `sail ps` أن `laravel.test` و `pgsql` و `redis` قيد التشغيل.

يُظهر `/mcp` في Claude Code أن `laravel-boost` متصل.

الملف `docs/plans/day1-domain.md` موجود ومُودَع، ويذكر سبعة جداول لشحنة بالضبط إضافة إلى العمود `role` في `users`.

لا يزال `sail artisan test` يُظهر **passed 2** (لم تغيّر أي شيفرة بعد).

الأمر `curl -s http://localhost/up` يُرجع HTTP 200 (مسار فحص السلامة المدمج في Laravel):

```
curl -s -o /dev/null -w "%{http_code}\n" http://localhost/up
```

```
200
```

الجزء 2 — التعدادات، والنماذج، والبيانات (14:15–15:45)

الخطوة 5 — كتابة التعدادات يدوياً (20 دقيقة)

ماذا ولماذا. تمنحك تعدادات PHP المدعومة مجموعة مغلقة من القيم تتفق عليها قاعدة البيانات والشيفرة والاختبارات جميعاً. ستكتبها بنفسك لتفهم بالضبط ما الذي يبني الوكيل فوقه.

> **تحذير**
>
> **يدوياً.** اكتب التعدادات الأربعة كلها بنفسك. استخدم الوكيل فقط *لمراجعتها* بعد ذلك.

أنشئ `app/Enums/ShipmentStatus.php`:

```php
<?php

namespace App\Enums;

enum ShipmentStatus: string
{
    case Pending = 'pending';
    case Confirmed = 'confirmed';
    case Assigned = 'assigned';
    case PickedUp = 'picked_up';
    case InTransit = 'in_transit';
    case Delivered = 'delivered';
    case Cancelled = 'cancelled';

    public function canTransitionTo(self $next): bool
    {
        // You will implement this in Step 6, test-first.
        return false;
    }
}
```

اكتب الآن، بالأسلوب نفسه:

- `app/Enums/VehicleType.php` بالقيم `van` و`box_truck` و`flatbed` و`reefer`.
- `app/Enums/VehicleStatus.php` بالقيم `available` و`busy` و`maintenance`.
- `app/Enums/UserRole.php` بالقيم `customer` و`dispatcher` و`admin`.

تحقّق من أن PHP يستطيع تحليلها:

```bash
sail artisan tinker --execute="dump(App\Enums\VehicleType::cases());"
```

يُفترض أن ترى أربع حالات من `App\Enums\VehicleType` مطبوعة.

الخطوة 6 — `canTransitionTo` بالاختبار أولاً (15 دقيقة)

ماذا ولماذا. دورة حياة الشحنة هي أهم قاعدة في شحنة: `pending → confirmed → assigned → picked_up → in_transit → delivered`، ولا يُسمح بالحالة `cancelled` إلا من `pending` أو `confirmed` أو `assigned`. قاعدة كهذه تستحق اختباراً يسرد كل انتقال مسموح وممنوع. ستجعل الوكيل يكتب **الاختبار أولاً** (Test-First)، وتشغّله ليفشل (أحمر)، ثم تنفّذ الشيفرة.

🤖 **اسأل وكيلك الذكي**

طلب للوكيل

اعرض عليّ خطة أولاً، ثم انتظر موافقتي.

اكتب اختبار وحدة بـ Pest في tests/Unit/ShipmentStatusTest.php للدالة App::canTransitionTo(). القواعد: - مسموح: pending->confirmed، confirmed->assigned، assigned->picked_up، picked_up->in_transit، in_transit->delivered. - مسموح: pending->cancelled، confirmed->cancelled، assigned->cancelled. - كل ما عدا ذلك ممنوع، بما في ذلك البقاء في الحالة نفسها، وتخطّي الخطوات، والرجوع إلى الخلف، وأي انتقال من delivered أو cancelled. استخدم مجموعات بيانات (Datasets) في Pest بحيث تُسرد صراحةً كل الأزواج المسموحة ومجموعة ممثِّلة من الأزواج الممنوعة. لا تغيّر ShipmentStatus.php بعد. ثم شغّل sail artisan test -filter=ShipmentStatus وأرني أنه يفشل.

بالإنجليزية:

```
First show me a plan, then wait for my approval.

Write a Pest unit test at tests/Unit/ShipmentStatusTest.php for
App\Enums\ShipmentStatus::canTransitionTo(). Rules:
- Allowed: pending→confirmed, confirmed→assigned, assigned→picked_up,
  picked_up→in_transit, in_transit→delivered.
- Allowed: pending→cancelled, confirmed→cancelled, assigned→cancelled.
- Everything else is forbidden, including staying in the same status,
  skipping steps, going backwards, and any move out of delivered or cancelled.
Use Pest datasets so every allowed pair and a representative set of forbidden
pairs are listed explicitly. Do NOT change ShipmentStatus.php yet.
Then run `sail artisan test --filter=ShipmentStatus` and show me it fails.
```

ما الذي تفحصه.

- جيد: مجموعة بيانات بالأزواج المسموحة الثمانية، ومجموعة بيانات للأزواج الممنوعة تتضمن `delivered → cancelled` و`picked_up → cancelled` و`pending → assigned` (تخطٍّ) و`assigned → pending` (رجوع إلى الخلف).
- جيد: يفشل الاختبار في الأزواج المسموحة (لأن الدالة المؤقتة (Stub) تُرجع `false`). الاختبار الذي ينجح مع دالة مؤقتة تُرجع `false` دائماً لا يختبر المسار المسموح.
- علامة خطر: الوكيل «يتطوّع» وينفّذ `canTransitionTo` أيضاً. ارفض هذا التغيير: لقد طلبت الأحمر أولاً.

شغّله بنفسك:

```
sail artisan test --filter=ShipmentStatus
```

```
  FAILED  Tests\Unit\ShipmentStatusTest > it allows valid transitions with ...
  Tests:    8 failed, 12 passed
```

(تعتمد الأرقام الدقيقة على عدد الأزواج الممنوعة في مجموعة البيانات؛ تنجح الأزواج الممنوعة لأن الدالة المؤقتة تُرجع `false`.)

نفّذ الدالة الآن. جرّبها **يدوياً** أولاً لمدة 5 دقائق باستخدام تعبير `match`. إذا علقت، فاطلب من الوكيل:

🤖 اسأل وكيلك الذكي

طلب للوكيل

نفّذ ShipmentStatus::canTransitionTo() بحيث تنجح الاختبارات في tests/Unit/ShipmentStatusTest.php. استخدم تعبير match واحداً على $this. لا تعدّل ملف الاختبار. شغّل الاختبار واعرض النتيجة.

بالإنجليزية:

```
Implement ShipmentStatus::canTransitionTo() so that the tests in
tests/Unit/ShipmentStatusTest.php pass. Use a single match expression on
$this. Do not modify the test file. Run the test and show the result.
```

يبدو التنفيذ الصحيح هكذا:

```php
public function canTransitionTo(self $next): bool
{
    return match ($this) {
        self::Pending => in_array($next, [self::Confirmed, self::Cancelled], true),
        self::Confirmed => in_array($next, [self::Assigned, self::Cancelled], true),
        self::Assigned => in_array($next, [self::PickedUp, self::Cancelled], true),
        self::PickedUp => $next === self::InTransit,
        self::InTransit => $next === self::Delivered,
        self::Delivered, self::Cancelled => false,
    };
}
```

```
sail artisan test --filter=ShipmentStatus
```

```
  Tests:    20 passed
```

تحذير

علامة خطر: إذا عدّل الوكيل الاختبار ليجعله ينجح (بحذف زوج ممنوع، أو قلب توقّع)، فتلك أشيع طريقة «يغشّ» بها الوكلاء. شغّل دائماً `git diff tests/` بعد أي خطوة تنفيذ.

أودِع التغييرات:

```
git add app/Enums tests/Unit/ShipmentStatusTest.php
git commit -m "feat: shipment status enum with transition rules"
```

الخطوة 7 — ملفات الترحيل والنماذج مع الوكيل (25 دقيقة)

ماذا ولماذا. يتولى الوكيل الآن العمل الكثيف، مسترشداً بالخطة التي وافقت عليها. اطلب **فروقاً صغيرة**: ملفات الترحيل أولاً، ثم مراجعتها، ثم النماذج.

🤖 **اسأل وكيلك الذكي**

طلب للوكيل

اتبع docs/plans/day1-domain.md بدقة. اعرض عليّ أولاً قائمة الملفات التي ستنشئها، ثم انتظر الموافقة.

الخطوة A فقط: أنشئ ملفات الترحيل، بترتيب الاعتماديات، باستخدام sail artisan make:migration. المتطلبات: - foreignId()->constrained() لكل مفتاح أجنبي، مع سلوك الحذف المذكور في الخطة. - decimal('price_lyd', 10, 2) للمبالغ المالية. لا أعداد عشرية عائمة (float). - فهرس فريد على shipments.quote_id shipments.referenceg cities.codeg، وعلى (from_city_id, to_city_id) في city_distances. - ملف ترحيل يضيف role إلى users بالقيمة الافتراضية 'customer'. - أعمدة التعداد تستخدم $table->enum() بقيم مأخوذة من تعداد PHP، مثل array_column(VehicleType::cases(), 'value'). لا تكتب القائمة يدوياً أبداً. لا تنشئ النماذج بعد. عند الانتهاء، شغّل sail artisan migrate:fresh واعرض عليّ المخرجات.

بالإنجليزية:

```
Follow docs/plans/day1-domain.md exactly. First show me the list of files you
will create, then wait for approval.

Step A only: create the migrations, in dependency order, using
`sail artisan make:migration`. Requirements:
- foreignId()->constrained() for every foreign key, with the on-delete
  behaviour from the plan.
- decimal('price_lyd', 10, 2) for money. No floats.
- unique index on shipments.quote_id, shipments.reference, cities.code,
  and (from_city_id, to_city_id) on city_distances.
- A migration that adds `role` to users with default 'customer'.
- Enum columns use $table->enum() with the values taken from the PHP enum,
  e.g. array_column(VehicleType::cases(), 'value'). Never hard-code the list.
Do not create models yet. When done, run `sail artisan migrate:fresh`
and show me the output.
```

راجع ملفات الترحيل بنفسك. افتح كل ملف جديد في `database/migrations/`. هذه هي الخطوة التي تضرب فيها «المزالق» التي تتناولها الدورة بأقسى صورها: ملف الترحيل الذي لا يُراجَع يصبح مخططاً دائماً.

- جيد: الطوابع الزمنية مرتّبة بحيث يُنفَّذ `cities` قبل `city_distances`، و`carriers` قبل `vehicles`، و`quotes` قبل `shipments`.
- جيد: دوال `down()` تحذف بالضبط ما أنشأته `up()`.
- علامة خطر: `$table->float('price_lyd')` أو `$table->double(...)`.
- علامة خطر: `$table->enum(...)` بقائمة قيم مكتوبة يدوياً بدلاً من `array_column(SomeEnum::cases(), 'value')`؛ إذ سيتباعد تعداد PHP والعمود مع الوقت.

- علامة خطر: جداول أو أعمدة إضافية غير موجودة في الخطة.
- افحص: يجب أن يطابق سلوك الحذف للعمود `shipments.user_id` ما في خطتك. يستخدم الحل المرجعي `cascadeOnDelete()` (حذف المستخدم يحذف شحناته)؛ أما `restrict` فيحافظ على سجل الشحنات بأمان أكبر في بيئة الإنتاج. كلاهما مقبول ما دامت خطتك تنص عليه وتستطيع تفسيره.

افحص المخطط الحقيقي، بمعزل عن ملخص الوكيل:

```
sail psql -c '\d shipments'
```

```
      Column        |              Type              | Nullable
--------------------+--------------------------------+----------
 id                 | bigint                         | not null
 reference          | character varying(10)          | not null
 user_id            | bigint                         | not null
 quote_id           | bigint                         | not null
 vehicle_id         | bigint                         |
 ...
 price_lyd          | numeric(10,2)                  | not null
Indexes:
    "shipments_pkey" PRIMARY KEY, btree (id)
    "shipments_quote_id_unique" UNIQUE CONSTRAINT, btree (quote_id)
    "shipments_reference_unique" UNIQUE CONSTRAINT, btree (reference)
```

والآن النماذج:

🤖 اسأل وكيلك الذكي

طلب للوكيل

الخطوة B: أنشئ نماذج Eloquent: City، CityDistance، Carrier، Vehicle، Quote، Shipment، ShipmentEvent وفق docs/plans/day1-domain.md. - استخدم الدالة casts() في Laravel 12. حوّل أعمدة التعداد إلى التعدادات في App، price_lyd إلى 'decimal:2'، والتواريخ والأوقات إلى 'datetime'. - أضف علاقات belongsTo / hasMany الموصوفة في الخطة، مع أنواع الإرجاع. أضف تحويل User->role والعلاقتين User->shipments() وUser->quotes(). - استخدم $fillable صراحةً، ولا تستخدم $guarded = [] أبداً. - Shipment: ولّد reference في حدث النموذج creating إذا كان فارغاً، بالصيغة "SHN-" + 6 حروف/أرقام كبيرة (Str::upper(Str::random(6))). فرق صغير، ولا ملفات أخرى. ثم شغّل sail artisan test.

بالإنجليزية:

```
Step B: create Eloquent models City, CityDistance, Carrier, Vehicle, Quote,
Shipment, ShipmentEvent following docs/plans/day1-domain.md.
- Use the Laravel 12 `casts()` method. Cast enum columns to the enums in
  App\Enums, price_lyd to 'decimal:2', datetimes to 'datetime'.
- Add belongsTo / hasMany relationships described in the plan, with return
  types. Add User→role cast and User→shipments() and User→quotes().
- Use $fillable explicitly, never $guarded = [].
- Shipment: generate `reference` in a `creating` model event if empty, as
  "SHN-" + 6 uppercase alphanumerics (Str::upper(Str::random(6))).
Small diff, no other files. Then run `sail artisan test`.
```

ما الذي تفحصه.

- جيد: `protected function casts(): array` (أسلوب Laravel 11 فما بعده)، لا الخاصية القديمة `protected $casts`. كلاهما يعمل، لكن `CLAUDE.md` لديك يطلب الأسلوب الجديد.
- جيد: `public function vehicle(): BelongsTo` مع الاستيراد من `Illuminate\Database\Eloquent\Relations`.
- علامة خطر: `$guarded = []`. فهذا يعطّل الحماية من الإسناد الجماعي (Mass Assignment).
- علامة خطر: أسماء علاقات لا تطابق الأعمدة (مثلاً `origin()` دون تحديد `'origin_city_id'` مفتاحاً أجنبياً، مما يجعل Eloquent يبحث عن `origin_id`).

أثبت أن العلاقات تعمل في Tinker. هذا فحص **يدوي**:

```
sail artisan tinker
```

```
> $c = App\Models\City::create(['name_en' => 'Tripoli', 'name_ar' => 'طرابلس', 'code' =>
      'TIP']);
> $c->code;
= "TIP"
> App\Enums\ShipmentStatus::from('pending')
      ->canTransitionTo(App\Enums\ShipmentStatus::Delivered);
= false
```

أودِع التغييرات:

```
git add database/migrations app/Models
git commit -m "feat: shahna domain migrations and models"
```

الخطوة 8 – المصانع والمُغذّي (20 دقيقة)

ماذا ولماذا. تجعل المصانع الاختبارات سهلة القراءة (`Shipment::factory()->create()`)؛ ويمنح المُغذّي الجميع البيانات الواقعية نفسها للعمل على واجهة البرمجة (API) غداً.

🤖 اسأل وكيلك الذكي

طلب للوكيل

خطّط أولاً، ثم انتظر الموافقة.

1. أنشئ مصانع لكل نموذج من نماذج شحنة. يجب أن تنشئ المصانع سجلات مرتبطة صالحة (مثلاً ShipmentFactory تنشئ Userg Quote والمدن الخاصة بها). أضف حالات المصانع: dispatcher()/admin() في UserFactory، و status(ShipmentStatus $status) في ShipmentFactory.
2. أنشئ CitySeeder بهذه المدن الثماني بالضبط (code، name_en، name_ar): TIP Tripoli طرابلس، BEN Benghazi بنغازي، MRA Misrata مصراتة، SEB Sebha سبها، ZAW Zawiya الزاوية، TOB Tobruk طبرق، SRT Sirte سرت، GHA Ghat غات. في المُغذّي نفسه، أضف مسافات الطرق بالكيلومتر لجميع أزواج المدن الثمانية والعشرين، مخزّنة في الاتجاهين (بالقيمة نفسها). ضع المسافات في ثابت مصفوفة واحد لأتمكّن من مراجعتها.
3. أنشئ FleetSeeder بأربع شركات نقل ومركبتين من كل نوع في كل مدينة (capacity_kg: van 1500، box_truck 8000، flatbed 20000، reefer 10000).
4. في DatabaseSeeder، استدعِ CitySeeder وFleetSeeder، ثم أنشئ ثلاثة مستخدمين تجريبيين بكلمة المرور "password": customer@shahna.test (customer)، dispatcher@shahna.test (dispatcher)، admin@shahna.test (admin).
5. يجب أن تكون المُغذّيات متساوية الأثر عند التكرار (updateOrCreate / firstOrCreate). شغّل sail artisan migrate:fresh -seed وsail artisan test.

بالإنجليزية:

```
Plan first, then wait for approval.

1. Create factories for every Shahna model. Factories must create valid
   related records (e.g. ShipmentFactory creates its own Quote, User, cities).
   Add factory states: UserFactory dispatcher()/admin(), and
   ShipmentFactory status(ShipmentStatus $status).
2. Create CitySeeder with exactly these 8 cities (code, name_en, name_ar):
   TIP Tripoli طرابلس, BEN Benghazi بنغازي, MRA Misrata مصراتة,
   SEB Sebha سبها, ZAW Zawiya الزاوية, TOB Tobruk طبرق, SRT Sirte سرت,
   GHA Ghat غات.
   In the same seeder, add road distances in km for all 28 city pairs, stored
   in both directions (same value). Put the distances in one array constant
   so I can review them.
3. Create FleetSeeder with 4 carriers and 2 vehicles of every type in every
   city (capacity_kg: van 1500, box_truck 8000, flatbed 20000, reefer 10000).
4. In DatabaseSeeder, call CitySeeder and FleetSeeder, then create three demo
   users with password "password": customer@shahna.test (customer),
   dispatcher@shahna.test (dispatcher), admin@shahna.test (admin).
5. Seeders must be idempotent (updateOrCreate / firstOrCreate).
Run `sail artisan migrate:fresh --seed` and `sail artisan test`.
```

تحقّق من المسافات يدوياً. النماذج اللغوية الكبيرة (LLMs) ضعيفة في الأرقام الجغرافية. اختر ثلاثة أزواج وتحقّق من معقوليتها بمقارنتها بخريطة (مسافات الطرق التقريبية: طرابلس-مصراتة نحو 210 كم، وطرابلس-بنغازي نحو 1,000 كم، وطرابلس-سبها نحو 780 كم). إذا كانت قيمة ما بعيدة جداً عن الواقع، فأصلحها **يدوياً** في مصفوفة المُغذّي. أي قيمة ضمن هامش 10% تقريباً مقبولة في هذه الدورة.

ما الذي تفحصه.

- جيد: 8 مدن، و56 صفاً للمسافات (8 × 7 أزواج مرتّبة).
- جيد: تشغيل `CitySeeder` و`FleetSeeder` مرتين لا يُنشئ تكرارات. (المستخدمون التجريبيون في `DatabaseSeeder` يستخدمون `create()`، لذا يفشل تشغيل `db:seed` الكامل على قاعدة بيانات مُغذّاة مسبقاً بسبب تفرّد البريد الإلكتروني؛ استخدم `migrate:fresh --seed` لإعادة تغذية كل شيء.)
- علامة خطر: يُغذّي الوكيل جدول دول فيه «Libya»، أو عناوين، أو سائقين. احذف أي شيء غير موجود في الخطة.
- علامة خطر: أسماء عربية تظهر مشوّهة. تأكّد من أن ترميز الملف UTF-8.

شغّل عمليات العدّ بنفسك:

```
sail artisan migrate:fresh --seed
sail artisan tinker --execute="dump(App\Models\City::count(), App\Models\CityDistance::count(),
        App\Models\Vehicle::count());"
```

```
8
56
64
```

(64 = 8 مدن × 4 أنواع مركبات × مركبتان.)

شغّل مُغذّيَي المدن والأسطول مرة ثانية وتأكّد من أن الأعداد لم تتغير:

```
sail artisan db:seed --class=CitySeeder
sail artisan db:seed --class=FleetSeeder
sail artisan tinker --execute="dump(App\Models\City::count(), App\Models\CityDistance::count(),
        App\Models\Vehicle::count());"
```

أخيراً، اطلب من الوكيل مراجعة عمل يومه، ثم احكم أنت على المراجعة:

🤖 **اسأل وكيلك الذكي**

طلب للوكيل

راجع الفرق بين الفرع my-day1 والوسم day1-start (git diff day1-start). قارنه بالملف docs/plans/day1-domain.md. اذكر: (1) أي شيء في الشيفرة غير موجود في الخطة، (2) أي شيء في الخطة مفقود من الشيفرة، (3) أي مبالغ مالية مخزّنة بالنوع float، وأي قيد تفرّد مفقود، وأي مفتاح أجنبي مفقود. لا تُصلح أي شيء: اكتفِ بالتقرير.

بالإنجليزية:

```
Review the diff of branch my-day1 against day1-start (git diff day1-start).
Compare it with docs/plans/day1-domain.md. List: (1) anything in the code
that is not in the plan, (2) anything in the plan that is missing in the code,
(3) any money stored as float, any missing unique constraint, any missing
foreign key. Do not fix anything; just report.
```

نقطة تحقق

نقطة التحقق 2 (15:45)

```
sail artisan migrate:fresh --seed
sail artisan test
```

```
   PASS  Tests\Unit\ShipmentStatusTest
  ✓ it allows valid transitions with ...
  ✓ it forbids invalid transitions with ...
  ...
  Tests:    23 passed
```

يحتوي الحل المرجعي (`day1-end`) على **23** اختباراً. يعتمد العدد الدقيق لديك على حجم مجموعة البيانات؛ المهم هو **صفر حالات فشل**.

فحص سريع للبيانات عبر التطبيق (لا توجد واجهة برمجة بعد؛ هذا موعده غداً):

```
sail artisan tinker --execute="echo App\Models\City::where('code','BEN')->first()
      ->toJson(JSON_UNESCAPED_UNICODE);"
```

```
{"id":2,"name_en":"Benghazi","name_ar":"بنغازي","code":"BEN","created_at":"...","updated
       _at":"..."}
```

أودِع التغييرات وتكون قد انتهيت:

```
git add -A
git commit -m "feat: factories and seeders"
```

إذا واجهتك صعوبة

الحل المرجعي لليوم هو الوسم `day1-end` . قارن عملك به بدلاً من نسخه دون فهم:

```
# What files differ between your branch and the reference?
git diff --stat day1-end

# Look at one file in the reference without changing your branch
git show day1-end:app/Enums/ShipmentStatus.php

# Diff one file
git diff day1-end -- database/seeders/CitySeeder.php
```

إذا تعذّر إصلاح فرعك، فابدأ من جديد انطلاقاً من الحل المرجعي (يبقى عملك محفوظاً في `my-day1`):

```
git stash
git switch -c my-day1-reset day1-end
sail artisan migrate:fresh --seed
sail artisan test
```

غداً يبدأ الجميع من `day1-end`، فلا يتخلّف أحد.

أهداف إضافية

1. **قيود الفحص في قاعدة البيانات (Check Constraints).** أضف ملف ترحيل يستخدم `DB::statement` ليفرض `weight_kg > 0` و`price_lyd ⩾ 0` على `quotes` و`shipments`، و`from_city_id <> to_city_id` على `city_distances`. اكتب اختبار Pest يثبت أن قاعدة البيانات ترفض الوزن السالب.
2. **حارس على مستوى النموذج.** أضف `Shipment::transitionTo(ShipmentStatus $next, ?string $note = null)` التي ترمي استثناءً خاصاً بالنطاق (Domain Exception) عندما تكون `canTransitionTo` خاطئة، وإلا فإنها تحدّث الحالة وتسجّل `ShipmentEvent` ضمن معاملة (Transaction) واحدة في قاعدة البيانات. بالاختبار أولاً.
3. **تسميات عربية.** أضف إلى كل تعداد الدالة `label(string $locale)` التي تُرجع تسمية بالإنجليزية أو بالعربية. اطلب من الوكيل كتابتها، ثم راجع كل نص عربي مع متحدث أصلي بالعربية في القاعة.
4. **اكتب أمراً مائلاً مخصصاً (Custom Slash Command).** أنشئ `.claude/commands/review-migration.md` يحتوي طلباً قابلاً لإعادة الاستخدام يراجع ملفات الترحيل مقارنةً بالخطة. استخدمه على ملفات الترحيل الخاصة بك.

التأمل

أجب عن كل سؤال في جملتين أو ثلاث، وأحضر إجاباتك إلى الاجتماع الصباحي القصير (Standup) غداً.

1. اذكر شيئاً واحداً أخطأ فيه الوكيل اليوم أو أضافه دون أن يُطلب منه. كيف لاحظته، وماذا كان سيحدث لو لم تلاحظه؟
2. ما الفحوص التي أجريتها *خارج* الوكيل (Tinker، `psql`، `git diff`، خريطة)؟ أيّها اكتشف شيئاً لم يذكره ملخص الوكيل نفسه؟
3. انظر إلى ملف `CLAUDE.md` لديك. ما السطر الواحد الذي ستضيفه إليه، بناءً على تجربة اليوم، ليرتكب الوكيل أخطاء أقل غداً؟